

MongoDB Shell[Install mongosh](#)[Connect to a Deployment](#)[► Configure mongosh](#)[Run Commands](#)[► Perform CRUD Operations](#)[Run Aggregation Pipelines](#)[Client-Side Field Level
Encryption](#)[► Write Scripts](#)[► Snippets](#)[▼ Reference](#)[Compatibility Changes with
Legacy mongo Shell](#)[Data Types](#)[Methods](#)[▼ EJSON](#)[EJSON.parse\(\)](#)[EJSON.stringify\(\)](#)[.mongoshrc Configuration
File](#)[Options](#)[Docs Home](#) → [View & Analyze Data](#) → [MongoDB Shell](#)

EJSON.parse()

On this page**Syntax**[Command Fields](#)[Behavior](#)[Examples](#)[Format Input with EJSON.parse\(\)](#)[Use EJSON.parse\(\) from the command line](#)[Learn More](#)

The `EJSON.parse()` method converts string values to JSON.

Syntax

The `EJSON.parse()` method takes a string as input and an optional modifier that controls the output format.

```
EJSON.parse(string, [options])
```

Command Fields [🔗](#)

The `EJSON.parse()` method takes these fields:

On this page**Syntax**[Command Fields](#)[Behavior](#)[Examples](#)[Format Input with
EJSON.parse\(\)](#)[Use EJSON.parse\(\)
from the command
line](#)[Learn More](#)

Field	Type	Necessity	Description
<code>value</code>	string	Required	String <code>EJSON.parse()</code> transforms into JSON key-value pairs
<code>options</code>	string	Optional	Modifies output types. The only option is <code>{ relaxed: <boolean> }</code> .

Value	Effect
<code>true</code>	Returns native JavaScript types when possible
<code>false</code>	Prefer to return BSON types

Behavior

You can call `EJSON.parse()` from inside an interactive `mongosh` session or from the system command line using `--eval`.

Call `EJSON.parse()` from an interactive session:

```
EJSON.parse(string)
```

Call `EJSON.parse()` from the system command line:

```
mongosh --eval "EJSON.parse(string)"
```

Examples

To try these examples, first create the `sales` collection:

```
db.sales.insertMany( [
  { custId: 345, purchaseDate: ISODate("2023-07-04"), quantity: 4,
  { custId: 346, purchaseDate: ISODate("2023-07-12"), quantity: 3,
  { custId: 486, purchaseDate: ISODate("2023-08-01"), quantity: 9,
] )
```



Format Input with EJSON.parse()

`EJSON.parse()` accepts a string as input. For this example, use the `EJSON.stringify()` method to export the `sales` collection as a string.

```
let salesCollection = EJSON.stringify( db.sales.find().toArray() )
```



Use `EJSON.parse()` to format the string for methods like `db.collection.insertMany()` that expect JSON pairs:

```
db.salesRestored.insertMany( EJSON.parse( salesCollection ) )
```



- `EJSON.parse()` formats the values in `salesCollection` as JSON pairs.
- `db.salesRestored.insertMany()` uses the JSON pairs to create the `salesRestored` collection.

Use EJSON.parse() from the command line

To import string data from an external source such as a file or an API call, use `EJSON.parse()` with the `mongosh --eval` method.

For this example, save the `sales` collection as a file.

```
let salesCollection = EJSON.stringify( db.sales.find().toArray() )  
  
fs.writeFileSync( 'sales.json', salesCollection )
```



The code creates a file on your local system called `sales.json`. To import the file and create a new collection, exit `mongosh` and run an `--eval` operation from the command line.

```
# Note: This example is formatted to fit on the page.
```



```
mongosh --quiet \  
  --eval "db.salesFromFile.insertMany( \  
    EJSON.parse( fs.readFileSync( 'sales.json', 'utf8' ) ) )"

```

- `EJSON.parse()` takes a string as input. This example uses `fs.readFileSync()` to read the `sales.json` file as a string.
- `EJSON.parse()` formats the input string as JSON pairs.
- `db.salesFromFile.insertMany()` creates the `salesFromFile` collection from the JSON pairs.

Learn More

- [EJSON](#) documentation
- [Mozilla Developer Network JSON.parse](#) documentation